Transformer Scaling Law

1 Dense Transformer Scaling

We fit a Chinchilla-style loss model to the dense transformer sweep:

$$L(N, D) = E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}},$$

where N is the parameter count, D is the number of training tokens seen, and compute is approximated as

$$C = 6ND.$$

Using the 10-epoch constant-learning-rate sweep, the fitted dense law was:

$$L(N, D) \approx 3.86 \times 10^{-14} + \frac{7.10 \times 10^{12}}{N^{2.761}} + \frac{5.04 \times 10^{6}}{D^{1.302}}.$$

This implies the compute-optimal allocation:

$$N_{\text{opt}}(C) \approx 39.23 \left(\frac{C}{6}\right)^{0.320},$$

$$D_{\text{opt}}(C) \approx 0.0255 \left(\frac{C}{6}\right)^{0.680}.$$

Quantity	Value	Interpretation
E	3.86×10^{-14}	Near-zero irreducible loss floor
α	2.761	Model-size loss term falls quickly with N
β	1.302	Data/token loss term falls substantially with D
N exponent	0.320	Parameters grow slowly with compute
D exponent	0.680	Training tokens grow faster with compute

Table 1: Dense transformer fitted law and optimal allocation summary.

The main qualitative result is that the optimal allocation is data-heavy. For a $10\times$ increase in compute, the fitted law recommends roughly

$$10^{0.320} \approx 2.1\times$$

more parameters, but

$$10^{0.680} \approx 4.8\times$$

more training tokens. Thus, for this addition task, the dense model benefits from scaling both model size and data, but the fitted optimum allocates most new compute to more token exposure.

This differs from the classic language-model Chinchilla result, where model size and data size scale more evenly. A plausible explanation is that this is a small synthetic arithmetic task with a near-zero achievable loss. Once the model has enough capacity to represent the algorithm, additional training exposure can be more valuable than additional parameters. The result is also consistent with the observed frontier: small models dominate at low compute, then the frontier moves to larger models only once the token budget is large.

2 MoE Scaling

For the mixture-of-experts model, we distinguish between total parameters and active parameters:

$$N_{\text{total}} = \text{all stored parameters},$$

$$N_{\text{active}} = \text{parameters active for a token}.$$

The sweep used top-1 routing with four experts, so the MoE models have roughly three times as many stored parameters as active parameters:

$$\frac{N_{\text{total}}}{N_{\text{active}}} \approx 3.$$

The active compute proxy is

$$C_{\text{active}} = 6N_{\text{active}}D.$$

The fitted active-parameter MoE law was:

$$L(N_{\text{active}}, D) \approx 4.64 \times 10^{-18} + \frac{2.42 \times 10^{18}}{N_{\text{active}}^{4.000}} + \frac{4.77 \times 10^{6}}{D^{1.284}}.$$

This gives:

$$N_{\text{active,opt}}(C_{\text{active}}) \approx 203.61 \left(\frac{C_{\text{active}}}{6}\right)^{0.243},$$

$$D_{\text{opt}}(C_{\text{active}}) \approx 0.00491 \left(\frac{C_{\text{active}}}{6}\right)^{0.757}.$$

We also fit a stored-capacity view:

$$L(N_{\text{total}}, D) \approx 1.95 \times 10^{-13} + \frac{1.65 \times 10^{20}}{N_{\text{total}}^{4.000}} + \frac{4.77 \times 10^{6}}{D^{1.284}}.$$

The corresponding proxy optimum was:

$$N_{\text{total,opt}}(C_{\text{proxy}}) \approx 453.03 \left(\frac{C_{\text{proxy}}}{6}\right)^{0.243},$$

$$D_{\text{opt}}(C_{\text{proxy}}) \approx 0.00221 \left(\frac{C_{\text{proxy}}}{6}\right)^{0.757}.$$

The MoE result is rougher than the dense result. The most important warning is that α hit the upper bound of the fitting range:

$$\alpha = 4.000.$$

Law	N exponent	D exponent	Fit note
Dense	0.320	0.680	Data-heavy, but both terms identifiable
MoE active	0.243	0.757	Even more data-heavy
MoE total	0.243	0.757	Similar because N_{total} tracks N_{active}

Table 2: Compute-optimal exponents for dense and MoE fits.

This should not be interpreted as evidence that the true MoE model-size exponent is exactly 4. Rather, it means the available data does not cleanly identify the model-size term. The optimizer wanted the N-dependent loss penalty to vanish even faster than the allowed range. In practice, this says that the observed MoE loss variation is explained more clearly by D than by model size.

The empirical results support this caution. At the 10-epoch slice, dense models still win the best validation loss at each sample size. However, MoE models do show useful behavior at matched active-compute points. In particular, the extra stored expert capacity helps some smaller active models:

$$\text{200k samples, config 0: dense 0.531} \quad \text{vs. MoE 0.157},$$

$$\text{200k samples, config 1: dense 0.00683} \quad \text{vs. MoE 0.00472}.$$

This is the expected qualitative advantage of MoE: more stored capacity can help without increasing active compute by the same factor.

It's worth noting that the current MoE uses hard top-1 routing, no load-balancing loss, no expert-capacity control, and no auxiliary router loss. Those choices make optimization noisier and can hide clean scaling behavior. The task also saturates near zero loss, which makes log-space power-law fitting unstable at the high-compute end. For a cleaner MoE scaling law, the next steps would be to add load balancing, sweep more token budgets at high N, and fit frontier-adjacent points separately from obviously under-optimized points.

3 Pallas Kernel

The MoE feed-forward layer uses the sequence

$$h = \text{ragged\_dot}(x, W_{\text{up}}), \qquad h = \text{gelu}(h), \qquad y = \text{ragged\_dot}(h, W_{\text{down}}).$$

This materializes the hidden activation $h \in \mathbb{R}^{M \times F}$, where M is the number of routed tokens and F is the expert hidden width. We implemented a Pallas kernel that fuses the up projection, GELU, and down projection:

$$h_{\text{tile}} = xW_{\text{up,tile}}, \qquad h_{\text{tile}} = \text{gelu}(h_{\text{tile}}), \qquad \text{acc} \mathrel{+}= h_{\text{tile}} W_{\text{down,tile}}.$$

The fused kernel streams over tiles of F, so the full $[M, F]$ intermediate is never written to and reread from global memory.

The best-performing version was a per-expert Pallas backend. Instead of passing all expert weights into a single custom call, it launches one call per expert and passes only that expert's

$$W_{\text{up},e} \in \mathbb{R}^{D \times F}, \qquad W_{\text{down},e} \in \mathbb{R}^{F \times D}.$$

This reduced VMEM pressure compared with the compact all-expert kernel.

The strongest measured configuration was:

$$D = 512, \qquad \text{mlp\_ratio} = 8, \qquad F = 4096, \qquad E = 4, \qquad M = 6656.$$

The benchmark used batch size $B = 512$, sequence length $T = 13$, top-1 routing, and float32. The winning tile sizes were:

$$\text{BLOCK\_M} = 256, \qquad \text{BLOCK\_F} = 128.$$

Configuration	Baseline (ms)	Fused (ms)	Speedup	Max Abs. Diff.
$D = 384,\ r = 4$, compact Pallas	3.52	3.82	0.92×	0.0
$D = 384,\ r = 8$, compact Pallas	5.11	4.42	1.15×	0.0
$D = 512,\ r = 4$, compact Pallas	5.16	5.26	0.98×	0.0
$D = 512,\ r = 8$, per-expert Pallas	11.97	7.76	1.54×	0.0

Table 3: Full forward-pass timings for baseline ragged_dot and fused Pallas MoE FFN implementations. Here $r = \text{mlp\_ratio}$.

We also isolated the MoE FFN itself for the winning shape $(M, D, F, E) = (6656, 512, 4096, 4)$:

Routing Pattern	Baseline (ms)	Pallas (ms)	Speedup	Max Abs. Diff.
Balanced $[1664, 1664, 1664, 1664]$	1.572	0.993	1.58×	0.0
Skewed $[512, 1024, 2048, 3072]$	1.560	0.966	1.61×	0.0

Table 4: Isolated MoE FFN timings for the winning Pallas kernel shape.

The speedup appears when the hidden activation is large enough that avoiding its materialization dominates custom-kernel overhead. For $D = 512$ and $r = 8$, the baseline hidden activation has shape

$$[M, F] = [6656, 4096],$$

or

$$6656 \cdot 4096 = 27{,}262{,}976$$

float32 values, roughly 109 MB. The baseline path writes the up-projection output, reads it for GELU, writes the GELU result, and reads it again for the down projection. The fused kernel instead keeps only a tile such as $[256, 128]$ live inside the kernel and consumes it immediately.

At smaller shapes, especially $D = 384, r = 4$, the baseline `jax.lax.ragged_dot` remains competitive. The saved intermediate is not large enough to overcome routing, packing, custom-call, and scatter/gather overheads. For wider shapes, fusion should become more valuable, but the current implementations hit TPU VMEM limits if too much expert weight data is staged at once. Output-dimension tiling can make larger cases compile, but it recomputes the up projection and GELU for each output tile, which can erase the benefit.

The practical conclusion is that the fused Pallas MoE FFN is worthwhile once F is large relative to D. In our experiments, the clearest win was $D = 512, r = 8, F = 4096$, where the per-expert Pallas kernel achieved about 1.54× end-to-end forward speedup and about 1.6× isolated FFN speedup with exact float32 output parity.